Combination Sum (medium)

Problem Statement

Given an array of distinct positive integers `candidates` and a target integer `target`, return a list of all **unique combinations** of candidates where the chosen numbers sum to `target`. You may return the combinations in **any order**.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

Example 1:

```
Input: candidates = [2, 3, 6, 7], target = 7
Output: [[2, 2, 3], [7]]
Explanation: The elements in these two combinations sum up to 7.
```

Example 2:

```
Input: candidates = [2, 4, 6, 8], target = 10
Output: [[2,2,2,2,2], [2,2,2,4], [2,2,6], [2,4,4], [2,8], [4,6]]
Explanation: The elements in these six combinations sum up to 10.
```

Try it yourself

Try solving this question here:

Java

```java
import java.util.ArrayList;
import java.util.List;

class Solution {
  public List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> res = new ArrayList<>();
    // TODO: Write your code here
    return res;
  }
}

```

Test Case | Run Code Result | Stdout

[2,3,6,7]
7

Use Example Testcases | Run | Submit

← Previous

Introduction To Backtracking Pattern

Next →

Solution: Combination Sum

Mark as Completed